

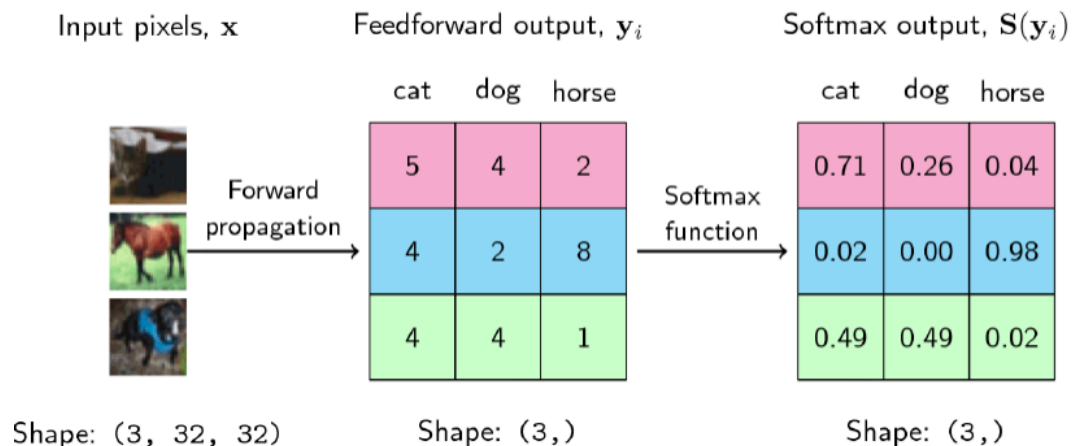
컴퓨터비전 개론 3주차 요약본(5. 26.~5. 29.)(3. 30.~4. 3.)

9. 다중 분류(Multinomial Classification) 1

9.1 다중 분류 모델 구조

- 딥러닝에서 다중 분류 모델의 기본 구조

- 입력층 (Input Layer) → 가중치(W) 및 활성화 함수(Softmax Function) → 출력층 (Output Layer)
- 출력층에서 Softmax 함수를 사용하여 각 클래스에 대한 확률을 계산



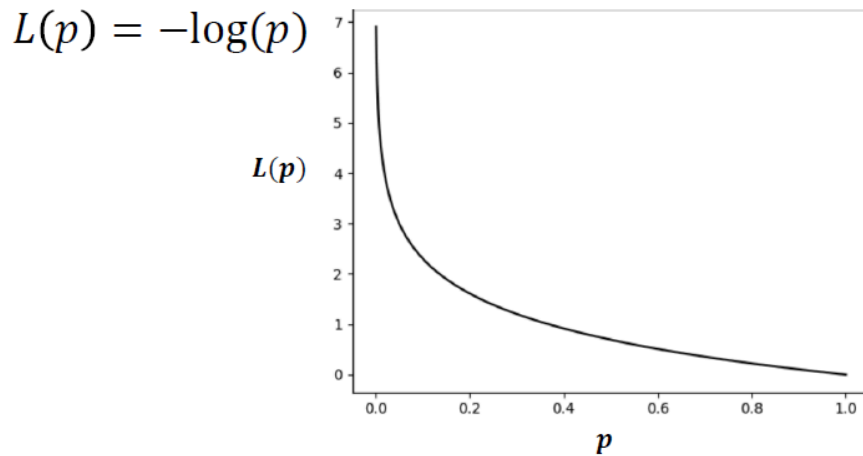
- Softmax 함수의 역할

- 다중 클래스 분류에서 각 클래스에 대한 확률을 제공
- 확률 값이 가장 높은 클래스를 최종 예측값으로 선택

9.2 Negative Log-Likelihood (NLL)

- NLL 개념

- 모델의 출력 확률과 실제 클래스 간의 차이를 측정하는 손실 함수
- 예측이 정확할수록 NLL 값이 작아짐



p : 모델이 예측한 해당 클래스의 확률값 NLL 값이 작을수록 모델의 예측 성능이 좋음

9.3 다중 분류에서 Negative Log-Likelihood (Multiclass NLL)

$$J(\theta) = - \sum_{i=1}^n y_i \log h_{\theta}(x_i)$$

- y_i : 실제 클래스 레이블 (One-hot Encoding 사용).
- $h_{\theta}(x_i)$: Softmax를 적용한 모델의 예측 확률값.

다중 클래스 분류에서 손실 함수로 사용 → 모델이 특정 클래스를 예측할 확률이 높을수록 손실값이 작아짐.

10. 다중 분류 2

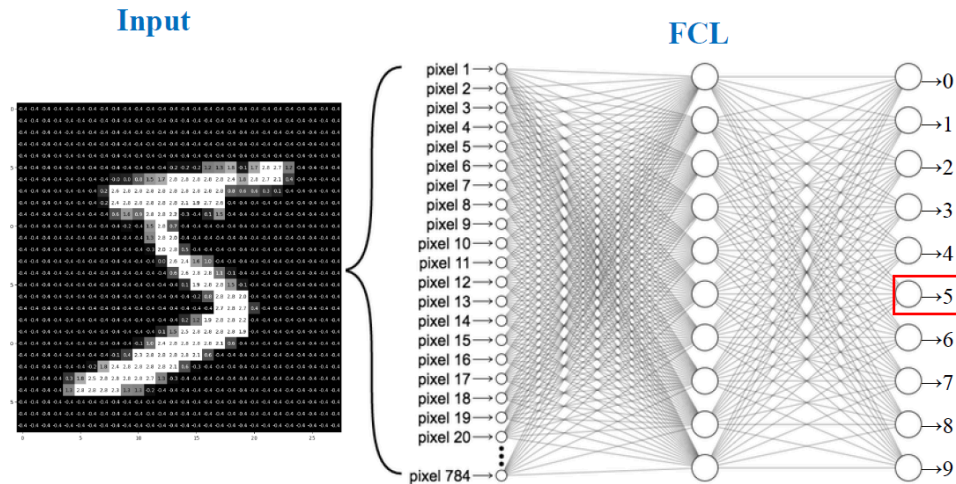
10.1 MNIST dataset에 대한 이해

- 이미지 인식, 분류 모델 학습 및 테스트용 데이터셋
- 손글씨로 작성된 0~9 숫자(10종)으로 구성
- 총 7만개의 데이터 (훈련용 이미지 6만개, 테스트용 이미지 1만개)
- 28×28 픽셀의 흑백 이미지

10.2 Fully connected layers를 이용한 MNIST dataset 분류

- Fully connected layers(FCL)

- MLP(다층 퍼셉트론)라고도 함
- CNN없이 학습 가능



- 입력: $28 \times 28 = 784$ 차원
- 출력: 10 클래스(0~9 숫자)
- 예제 코드
 - Torchvision dataset 모듈 사용
 - Torchvision transforms 를 활용한 이미지 데이터 전처리
 - Pytorch DataLoader를 이용한 Mini-batch 생성

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt

# 1. 데이터셋
# Torchvision transforms를 활용한 이미지 데이터 전처리
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

train_data = datasets.MNIST(root="./data", train=True, d
```

```

download=True, transform=transform)
test_data = datasets.MNIST(root="./data", train=False, d
ownload=True, transform=transform)

# Pytorch DataLoader를 이용한 Mini-batch 생성
train_loader = DataLoader(train_data, batch_size=64, shu
ffle=True)
test_loader = DataLoader(test_data, batch_size=64, shuff
le=False)

# 2. 모델
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(28*28, 256)
        self.fc2 = nn.Linear(256, 128)
        self.fc3 = nn.Linear(128, 10)

    def forward(self, x):
        x = x.view(-1, 28*28)
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return self.fc3(x)

model = MLP()

# 3. 손실 함수 & 옵티마이저
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 4. 학습 루프
epochs = 3
for epoch in range(epochs):
    model.train()
    total_loss = 0
    for images, labels in train_loader:
        optimizer.zero_grad()
        outputs = model(images)

```

```

        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
        print(f"Epoch [{epoch+1}/{epochs}], Loss: {total_loss/len(train_loader):.4f}")

# 5. 평가
model.eval()
correct = 0
total = 0
all_preds = []
all_images = []
with torch.no_grad():
    for images, labels in test_loader:
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
        # 일부 이미지와 예측 저장
        all_images.extend(images[:8]) # 배치에서 앞 8개만
        # 저장
        all_preds.extend(predicted[:8])
        break # 한 배치만 시각화에 사용
print(f"테스트 정확도: {100 * correct / total:.2f}%")

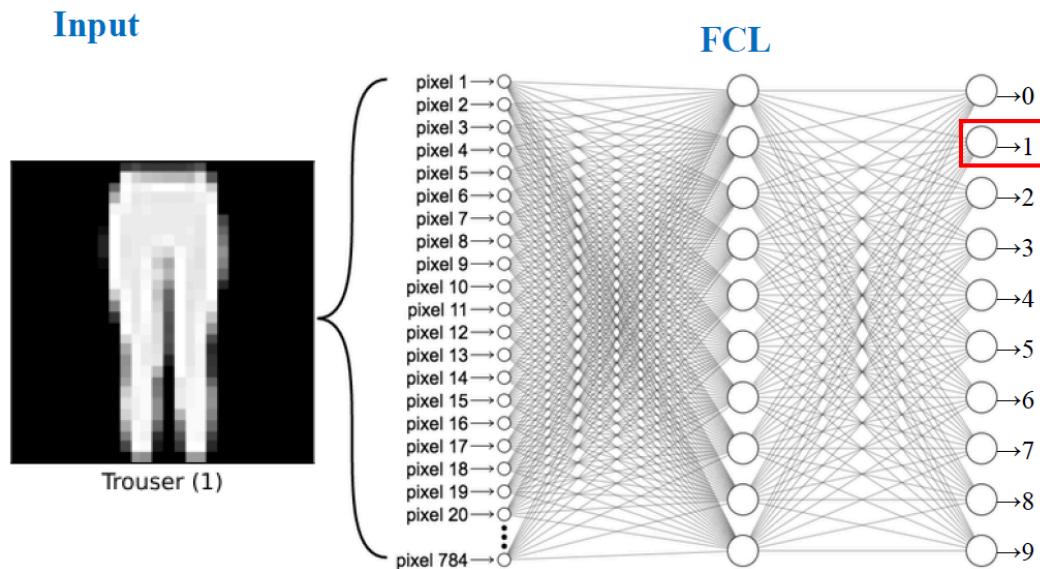
# 6. 결과 이미지 시각화
fig, axes = plt.subplots(2, 4, figsize=(10, 5))
for i, ax in enumerate(axes.flat):
    img = all_images[i].squeeze().numpy() # (28x28)
    ax.imshow(img, cmap="gray")
    ax.set_title(f"Pred: {all_preds[i].item()}")
    ax.axis("off")
plt.show()

```

10.3 Fully connected layers를 이용한 Fashion MNIST dataset 분류

- Fashion-MNIST: 의류 이미지(티셔츠, 바지, 신발 등) 28×28 픽셀

- MNIST와 구조는 같지만 난이도가 조금 더 높음
- 같은 네트워크 구조를 그대로 적용 가능



10.4 딥러닝 신경망의 구조

- **입력층 (Input Layer)**
 - 데이터가 모델로 처음 들어오는 층
 - 각 노드(뉴런) 가 입력 데이터의 한 특징(feature)을 나타냄
 - 예: 이미지(28×28) → 입력 노드 784개
- **은닉층 (Hidden Layer)**
 - 입력과 출력을 연결하며 패턴을 학습하는 층
 - 여러 개의 가중치(Weight) 와 활성화 함수(Activation Function) 로 구성
 - 비선형 관계를 학습하는 핵심 부분
 - 예: ReLU, Sigmoid, Tanh 등 사용
- **출력층 (Output Layer)**
 - 모델의 최종 결과를 내는 층
 - 회귀 → 1개의 노드 (예: 예측값)
 - 분류 → 클래스 수만큼 노드 (예: Softmax로 확률 출력)

10.4 Pytorch 데이터 전처리

구성요소	역할	예시
Dataset	데이터셋을 정의 (입력, 라벨 관리)	<code>torch.utils.data.Dataset</code>
Transforms	이미지/데이터 전처리, 변환	<code>torchvision.transforms</code>
DataLoader	배치 단위로 데이터 불러오기, 섞기, 병렬처리	<code>torch.utils.data.DataLoader</code>

10.5 배치 사이즈

구분	작은 배치	큰 배치
메모리 사용량	적음	많음
학습 속도	느림	빠름
기울기 안정성	불안정 (진동 가능)	안정적
일반화 성능	좋음 (과적합 ↓)	떨어질 수 있음 (과적합 ↑)
과적합 위험	낮음	높음
추천 상황	일반화 중시, 메모리 적을 때	대용량 데이터, 학습 속도 중요할 때

10.6 클래스 불균형

- 한 클래스의 데이터가 다른 클래스보다 **극도로 적거나 많은 현상**
 - 예: 스팸메일(1%) vs 일반메일(99%)
- 모델이 **다수 클래스에만 치우쳐 학습** → 소수 클래스 예측 성능 저하
- Accuracy가 높아도 실제 성능이 낮을 수 있음 (예: 모두 정상으로 예측)
- 클래스 불균형 해결 방법

방법	개념	특징 / 장점
가중 손실 (Weighted Loss)	손실 함수에 클래스별 가중치 부여 → 소수 클래스의 오차를 더 크게 반영	데이터 비율에 따라 자동 조정 가능모델 구조 변경 없음
SMOTE (Synthetic Minority Over-sampling Technique)	소수 클래스 데이터를 인공적으로 생성(보간) 하여 샘플 수 증가	단순 복제보다 다양성 확보, 과적합 완화
데이터 증강 (Data Augmentation)	이미지 회전·뒤집기 등으로 소수 클래스 데이터 다양화	특히 이미지/음성 데이터에 효과적
앙상블 기법 (Ensemble Methods)	여러 모델을 결합하여 소수 클래스 탐지 강화 (예: Bagging,	불균형 데이터에서도 안정적 성능 확보

방법	개념	특징 / 장점
	Boosting)	

11. CNN(Convolutional Neural Network)

11.1 Convolutional Neural Network (CNN) 개요

- **CNN이란?**
 - 이미지 인식 및 컴퓨터 비전에 특화된 신경망 모델
 - 이미지의 공간적 구조를 유지하면서 특징을 학습하는 능력을 가짐
 - 딥러닝 모델에서 영상 데이터 처리의 핵심 기술로 사용됨

11.2 주요 구성 요소

- **합성곱 층(Convolutional Layer)**
 - 필터(커널)를 사용해 특징 추출
 - 중요한 시각적 패턴 감지 (예: 가장자리, 윤곽)
- **풀링 층(Pooling Layer)**
 - 특징 맵 크기 축소 (Down Sampling)
 - 연산량 감소 및 중요한 정보 유지 (예: Max Pooling)
- **완전 연결 층(Fully Connected Layer, FC)**
 - 최종 분류 작업 수행
 - Flatten을 거쳐 1차원 형태로 변환 후 출력
- **활성화 함수(Activation Function)**
 - 비선형성을 추가하여 복잡한 패턴 학습
 - 주로 ReLU(Rectified Linear Unit) 사용
- **손실 함수(Loss Function) & 옵티마이저(Optimizer)**
 - 예측 값과 실제 값 차이를 줄이기 위해 사용
 - 분류 문제에서는 주로 Cross-Entropy Loss 활용

11.3 CNN의 연산 과정

1. 합성곱 연산(Convolution)

- 이미지의 특정 패턴(특징)을 추출하는 과정
- 필터(커널, Kernel)를 사용하여 이미지의 영역별 특징을 학습
- 합성곱 연산 후 특징 맵(Feature Map)이 생성됨

2. 스트라이드(Stride)

- 커널이 이동하는 간격을 의미
- 큰 값일수록 출력 크기가 작아지고, 작은 값일수록 세밀한 특징을 학습 가능

3. 풀링(Pooling)

- **맥스 풀링(Max Pooling)**: 가장 큰 값을 선택하여 대표값으로 사용
- **평균 풀링(Average Pooling)**: 해당 영역의 평균값을 선택

4. 패딩(Padding)

- 커널이 경계를 넘어가면서 데이터 손실을 방지하기 위해 0을 추가하는 기법
 - **유형**
 - **Valid Padding**: 패딩 없이 진행 (출력 크기가 작아짐)
 - **Same Padding**: 입력 크기를 유지하도록 패딩을 추가

5. Flattening Layer

- CNN의 마지막 합성곱 계층 출력을 1차원 배열로 변환하여 Fully Connected Layer로 전달

12. 튜닝 기법

12.1 깊은 신경망의 필요성

- 얇은 모델: 모서리·색 등 **저수준 특징**(모서리, 색상 등)만 포착 → 표현력 한계
- 깊은 모델: 저수준 → 중간 → 고수준 특징을 **계층적**으로 결합(눈·코·입 → 얼굴 전체)
- **AlexNet 이후** 더 깊은 VGG-19, ResNet(152층)로 발전
 - 실습 모델 구조
 - CNN_v2 설계 원칙($6 \times \text{Conv} + 3 \times \text{Pool}$)
 - 핵심: 풀링으로 이미지 크기 1/2 감소 시 → 채널 수 2배 증가
 - Block1: conv1-2 + pool1 → 32채널(기본 특징)

- Block2: conv3-4 + pool2 → 64채널(중간 특징)
- Block3: conv5-6 + pool3 → 128채널(고차원 특징)
 - 이유: 32채널 → 64채널 → 128채널 해상도 축소에 따른(정보 손실 최소화)
- 깊은 신경망은 얇은 모델보다 더 복잡·추상적 특징을 학습

12.2 최적화 함수 (Optimizer)

- 개념
 - 모델이 학습하면서 손실 함수(Loss Function)를 최소화하는 방향으로 파라미터(가중치(w: weight), 편향(b: bias))를 조정하는 과정
- 종류
 - **SGD (Stochastic Gradient Descent, 확률적 경사 하강법)**
 - 무작위로 한 개의 데이터 샘플을 사용하여 가중치 업데이트
 - 연산량 감소, 빠른 학습 가능하지만 느린 수렴 및 최적값 주변에서 진동 가능(지역 최소값, local minimum)
 - **SGD + 모멘텀(Momentum)**
 - 이전 기울기의 영향을 반영하여 관성을 가지며 이동
 - 진동을 줄이고 더 빠르게 수렴
 - 속도(velocity)를 고려하여 관성 효과를 부여
 - **Adam (Adaptive Moment Estimation)**
 - 1차 모멘트(기울기 평균)와 2차 모멘트(기울기 제곱 평균)를 활용하여 최적화 진행
 - 학습률 자동 조정 및 안정적인 초기 수렴을 보임
 - 대부분의 경우 좋은 성능을 보이며 가장 널리 사용됨

12.3 과적합(Overfitting)

- 과적합이란?
 - 모델이 학습 데이터에 과도하게 적응하여 일반화 성능이 저하되는 현상
 - 원인
 - 모델의 파라미터 수가 너무 많음
 - 데이터가 부족하여 noise까지 학습

- 학습 Epoch이 너무 많아 과학습 발생
- 과적합의 신호
 - 전형적 패턴: **train loss↓ 계속, val loss**가 어느 시점부터 **반등↑**
 - 즉, 검증 손실이 발생하기 시작하면 과적합 의심 징후

12.4 과적합 대응 방안

- 드롭아웃(Dropout)
 - 학습 중 일부 뉴런 **무작위 비활성화** → 특정 경로 의존 줄여 **일반화↑**
 - **Train 모드**: 예를 들어 $p=0.5$ 면 절반 0 처리 & 생존 뉴런 **스케일 보정**(값 2배)
 - **Eval 모드**: 적용 안 함(일관된 예측). `net.train()/eval()` 전환 필수
 - 실제 배치
 - 각 **MaxPool** 직후 배치
 - **분류기(Fully Connected layer)** 사이 배치
 - 깊어질수록 **비율 점진 증가**(예: $0.2 \rightarrow 0.3 \rightarrow 0.4$)
 - 주의 사항: 학습 시에만 적용! 평가 시에는 모든 뉴런 사용
 - `net.train()` - 드롭아웃 활성화
 - `net.eval()` - 드롭아웃 비활성화
- 배치 정규화 (Batch Normalization)
 - 미니배치 단위로 데이터의 분포를 정규화(평균:0, 분산:1) → 분포 안정화 및 학습 효율↑
 - 권장 위치: **Conv** → **BN** → **ReLU** (ReLU로 정보 소실 전에 분포 안정화)
 - 주의 사항: 같은 BN 인스턴스 **재사용 금지** → 채널 수 같아도 **별도 인스턴스** 필수!
 - 재사용하면 파라미터 공유로 학습 실패
- 데이터 증강(Data Augmentation)
 - 데이터 증강이 필요한 이유
 - 적은 데이터셋으로 인한 과적합 문제
 - 데이터 불균형 (예: 정상 90%, 비정상 10%)

- 원본 데이터를 변형하여 데이터 양 증가, 데이터 다양화로 **일반화 향상(다양한 상황에 대응 가능)**
- **제시 기법: RandomHorizontalFlip, RandomErasing**(사각 영역 제거로 구성 요소 편향 완화)

```
transform_train = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5), # 50% 좌우 반전
    transforms.ToTensor(),
    transforms.Normalize(0.5, 0.5),
    transforms.RandomErasing(p=0.5, scale=(0.02, 0.33)) #
영역 지우기
])
```